

Student 21 **Lakshmi Narayanan E** [192524249RD](#)

2 / 2 submitted

Q1. Smart Hospital Patient Registration System**CODE**

```
import java.util.*;

public class HospitalRegistrationSystem {
    static class Patient {
        String id;
        String name;
        String mobile;
        String department;
        String doctor;
        Patient(String id, String name, String mobile, String department, String doctor) {
            this.id = id;
            this.name = name;
            this.mobile = mobile;
            this.department = department;
            this.doctor = doctor;
        }
    }

    static ArrayList<Patient> patientList = new ArrayList<>();
    static HashMap<String, Patient> patientMap = new HashMap<>();
    static final int MAX_PATIENTS = 100;

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int choice;
        do {
            choice = Integer.parseInt(sc.nextLine().trim());
            switch (choice) {
                case 1:
                    registerPatient(sc);
                    break;
                case 2:
                    searchPatient(sc);
                    break;
                case 3:
                    displayAllPatients();
                    break;
                case 4:
                    departmentWiseCount();
                    break;
                case 5:
                    System.out.println("Exiting the System. Thank you!");
                    break;
                default:
                    System.out.println("Invalid Choice!");
            }
        } while (choice != 5);
        sc.close();
    }

    static void registerPatient(Scanner sc) {
        if (patientList.size() >= MAX_PATIENTS) {
            System.out.println("Maximum patient limit reached!");
            return;
        }
        String id = sc.nextLine().trim();
        if (patientMap.containsKey(id)) {
            System.out.println("Duplicate Patient ID! Registration Failed.");
            return;
        }
        String name = sc.nextLine().trim();

        if (!name.matches("[a-zA-Z ]+")) {
```

```

        System.out.println("Invalid Name! Only alphabets allowed.");
        return;
    }
    String mobile = sc.nextLine().trim();
    if (!mobile.matches("\\d{10}")) {
        System.out.println("Invalid Mobile Number! Must be exactly 10 digits.");
        return;
    }
    String department = sc.nextLine().trim().toUpperCase();
    String doctor = sc.nextLine().trim();
    name = toProperCase(name);
    Patient p = new Patient(id, name, mobile, department, doctor);
    patientList.add(p);
    patientMap.put(id, p);
    System.out.println("Patient Registered Successfully.");
    System.out.println();
}

static void searchPatient(Scanner sc) {
    String id = sc.nextLine().trim();
    Patient p = patientMap.get(id);
    if (p == null) {
        System.out.println("Patient Not Found");
    } else {
        System.out.println("Patient Details");
        System.out.println();
        System.out.println("ID : " + p.id);
        System.out.println();
        System.out.println("Name : " + p.name);
        System.out.println();
        System.out.println("Mobile : " + p.mobile);
        System.out.println();
        System.out.println("Department : " + p.department);
        System.out.println();
        System.out.println("Doctor : " + p.doctor);
    }
}

static void displayAllPatients() {
    System.out.println("-----");
    System.out.println("ID Name Department");
    System.out.println("-----");
    for (Patient p : patientList) {
        System.out.println(p.id + " " + p.name + " " + p.department);
    }
    System.out.println();
}

static void departmentWiseCount() {
    HashMap<String, Integer> deptCount = new HashMap<>();
    for (Patient p : patientList) {
        deptCount.put(p.department, deptCount.getOrDefault(p.department, 0) + 1);
    }
    System.out.println("Department Count");
    System.out.println();
    for (Map.Entry<String, Integer> entry : deptCount.entrySet()) {
        System.out.println(entry.getKey() + " : " + entry.getValue());
        System.out.println();
    }
}

static String toProperCase(String str) {
    String[] words = str.trim().split("\\s+");
    StringBuilder result = new StringBuilder();
    for (String word : words) {

```

```
        if (word.length() > 0) {
            result.append(Character.toUpperCase(word.charAt(0)))
                .append(word.substring(1).toLowerCase())
                .append(" ");
        }
    }
    return result.toString().trim();
}
```

INPUT

```
1
P101
Rahul Kumar
9876543210
Cardiology
Dr.Arun
1
P102
Priya
9876543211
Ortho
Dr.Meena
3
4
2
P101
5
```

OUTPUT

(no output)

Q2. Online Library Book Management System**CODE**

```
import java.util.Scanner;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;

public class Main {
    static class Book {
        String id;
        String title;
        String author;
        String category;
        String publisher;

        Book(String id, String title, String author, String category, String publisher) {
            this.id = id;
            this.title = title;
            this.author = author;
            this.category = category;
            this.publisher = publisher;
        }
    }

    static String trimSafe(String s) {
        if (s == null) return "";
        return s.trim();
    }

    static String toTitleCase(String s) {
        s = trimSafe(s);
        if (s.length() == 0) return s;
        s = s.toLowerCase();
        StringBuilder out = new StringBuilder();
        boolean newWord = true;
        for (int i = 0; i < s.length(); i++) {
            char ch = s.charAt(i);
            if (ch == ' ') {
                out.append(' ');
                newWord = true;
            } else {
                if (newWord) {
                    if (ch >= 'a' && ch <= 'z') ch = (char) (ch - 'a' + 'A');
                    newWord = false;
                }
                out.append(ch);
            }
        }
        return out.toString().replaceAll(" +", " ");
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        ArrayList<Book> books = new ArrayList<>();
        HashMap<String, Book> bookById = new HashMap<>();
        HashSet<String> authors = new HashSet<>();
        int maxBooks = 500;

        while (true) {
            if (!sc.hasNextLine()) break;
            String line = sc.nextLine().trim();
            if (line.length() == 0) continue;

            int choice;
            try {
```

```

        choice = Integer.parseInt(line);
    } catch (Exception e) {
        continue;
    }

    if (choice == 6) break;

    if (choice == 1) {
        if (books.size() >= maxBooks) continue;
        String id = trimSafe(sc.nextLine());
        String title = trimSafe(sc.nextLine());
        String author = trimSafe(sc.nextLine());
        String category = trimSafe(sc.nextLine());
        String publisher = trimSafe(sc.nextLine());

        boolean ok = true;
        if (id.length() == 0) ok = false;
        if (title.length() == 0) ok = false;
        if (author.length() == 0) ok = false;
        if (category.length() == 0) ok = false;
        if (publisher.length() == 0) ok = false;
        if (bookById.containsKey(id)) ok = false;

        if (!ok) {
            System.out.println("Invalid Entry or Duplicate Book ID.");
            continue;
        }

        title = toTitleCase(title);
        category = category.toUpperCase();

        Book b = new Book(id, title, author, category, publisher);
        books.add(b);
        bookById.put(id, b);
        authors.add(author);

        System.out.println("Book Added Successfully.");

    } else if (choice == 2) {
        String id = trimSafe(sc.nextLine());
        Book b = bookById.get(id);
        if (b == null) {
            System.out.println("Book Not Available");
        } else {
            System.out.println("Book ID : " + b.id);
            System.out.println("Title : " + b.title);
            System.out.println("Author : " + b.author);
            System.out.println("Category : " + b.category);
            System.out.println("Publisher : " + b.publisher);
        }
    }

    } else if (choice == 3) {
        System.out.println("-----");
        System.out.println("ID\tTITLE\tAUTHOR\tCATEGORY");
        System.out.println("-----");
        for (int i = 0; i < books.size(); i++) {
            Book b = books.get(i);
            System.out.println(b.id + "\t" + b.title + "\t" + b.author + "\t" + b.category);
        }
    }

    } else if (choice == 4) {
        System.out.println("Unique Authors");
        for (String a : authors) {
            System.out.println(a);
        }
    }

```

```
    } else if (choice == 5) {
        HashMap<String, Integer> catCount = new HashMap<>();
        for (int i = 0; i < books.size(); i++) {
            Book b = books.get(i);
            String c = b.category;
            Integer old = catCount.get(c);
            if (old == null) catCount.put(c, 1);
            else catCount.put(c, old + 1);
        }
        System.out.println("Category-wise Book Count");
        for (String key : catCount.keySet()) {
            System.out.println(key + " : " + catCount.get(key));
        }
    }
}
sc.close();
}
```

INPUT

```
1
B101
Java Programming
Herbert Schildt
Programming
McGraw Hill
1
B102
Data Structures
Mark Allen
Computer Science
Pearson
3
4
5
2
B101
6
```

OUTPUT

(no output)